

Notes TER 2026

Lounès KEBDI

4 février 2026

1 Definitions

1.1 Prompt Design (PD) : The "Hard Prompts" Approach

This is what 99% of people call "Prompt Engineering".

The Concept : We search for the magical combination of words (discrete tokens) to obtain the desired result.

Mechanism :

- You write in natural language (English/French).
- The input is fixed : The model converts your words into fixed vectors via its "tokenizer" and pre-learned embedding table.
- No model weights change. No training (Backpropagation).
- For your project : This is what you do when you test : "You are a clinical expert..."

Limitation : It is sub-optimal. Humans try to guess how the model "thinks". Sometimes, adding the word "Abracadabra" improves the result by 2% without knowing why.

1.2 Prompt Learning (PL) : The "Soft Prompts" Approach

Here, we stop trying to talk to the model with human words. We speak to it directly in mathematical vectors.

The Concept : Instead of searching for the words "Translate this", we search for the numerical vector $[0.12, -0.45, \dots]$ that triggers translation. This is what we call a "Continuous Prompt".

Why "Continuous" ?

- In the model's dictionary (Vocabulary), words are "discrete" (word #45, word #46). We cannot have word #45.5.
- In vector space (inside the model's brain), we can navigate anywhere. "Prompt Learning" searches for the optimal point in this space, even if this point does not correspond to any real word in the dictionary.
- Result : The optimal prompt found by the machine is often a sequence of incomprehensible numbers for humans, but which perfectly activates the task.

1.3 Prompt Tuning (PT) : The Efficient Implementation

Prompt Tuning is a specific method (and the most popular) for doing Prompt Learning. It is a form of PEFT (Parameter-Efficient Fine-Tuning).

The Technical Mechanism :

1. We take a giant LLM (e.g., Llama-3) and freeze all its weights (Frozen Backbone). We don't touch them.
2. We add a small sequence of trainable vectors (e.g., 20 to 100 vectors) in front of your input.
3. We launch training (Backpropagation).
4. The error (Loss) updates ONLY these small vectors added at the beginning.

The difference with classical Fine-Tuning :

- **Full Fine-Tuning** : We modify the 7 billion parameters of the model. (Heavy, expensive, risk of forgetting).
- **Prompt Tuning** : We modify only a few thousand parameters added as a prefix. The model remains intact.

Analogy :

- **Full Fine-Tuning** : You operate on the doctor's brain to teach them a new surgery.
- **Prompt Design** : You shout instructions at the doctor.
- **Prompt Tuning** : You slide a mathematical "post-it note" directly onto the doctor's visual cortex that modifies their perception of the task.

2 Prompt Design Techniques Summary

This section summarizes the main techniques used in Prompt Design (Hard Prompts) to improve model performance without modifying the model weights.

2.1 Zero-shot Prompting

Definition : Zero-shot prompting is the most basic form of prompt engineering where the model is asked to perform a task without any prior examples or demonstrations.

How it works :

- The user provides only the task description and the input query.
- The model relies entirely on its pre-trained knowledge to generate the response.
- No examples are given to guide the model's behavior.

Example :

Prompt: "Classify the following statement as Entailment, Contradiction, or Neutral: [statement]"

Advantages :

- Simple and straightforward to implement.
- No need to prepare example datasets.
- Fast inference time.

Limitations :

- Performance depends heavily on the model's pre-training.
- May struggle with complex or domain-specific tasks.
- Less reliable than few-shot approaches for nuanced tasks.

2.2 Few-shot Prompting

Definition : Few-shot prompting provides the model with a small number of example demonstrations (typically 2-10 examples) showing the desired input-output format before presenting the actual task.

How it works :

- The prompt includes several example pairs of (input, output).
- These examples serve as a "template" for the model to follow.
- The model learns the pattern from these examples and applies it to the new query.
- This technique tends to be more performant for multiple-choice questions (QCM) and structured tasks.

Example :

Prompt:

"Patient: I'm 45 years old.

Critère: Age $\geq$ 18 years.

Label: Entailment

Patient: I'm pregnant.

Critère: Pregnant women are excluded.

Label: Contradiction

Patient: [new patient statement]

Critère: [new criterion]

Label:"

Advantages :

- Significantly improves performance over zero-shot for many tasks.
- Helps the model understand the expected format and style.
- Particularly effective for classification and structured output tasks.

Limitations :

- Requires preparing example datasets.
- Token consumption increases with the number of examples.
- Quality of examples directly impacts performance.
- May hit context length limits with many examples.

2.3 Chain-of-Thought (CoT) Prompting

Definition : Chain-of-Thought prompting is one of the most popular techniques in Prompt Design. It asks the model to explicitly show its reasoning process step-by-step before providing the final answer.

How it works :

- Instead of asking for a direct answer, the prompt instructs the model to "think step by step" or "show your reasoning".
- The model breaks down complex problems into intermediate reasoning steps.
- Each step builds upon the previous one, creating a logical chain of reasoning.
- The final answer is derived from this explicit reasoning process.

Example :

Prompt: "Analyze the following patient statement step by step:

Patient: I'm a 55-year-old woman with osteoporosis.

Critère: Adult patients with bone density issues.

Step 1: Identify patient characteristics...

Step 2: Match against criteria...

Step 3: Determine relationship...

Final Label: Entailment"

Advantages :

- Dramatically improves performance on complex reasoning tasks.
- Makes the model's reasoning process transparent and interpretable.
- Helps catch errors in the reasoning chain.
- Particularly effective for mathematical, logical, and multi-step problems.

Limitations :

- Increases token consumption significantly.
- May generate verbose or unnecessary reasoning steps.
- Requires careful prompt design to guide the reasoning process effectively.

Variants :

- **Zero-shot CoT :** Simply adding "Let's think step by step" to a zero-shot prompt.
- **Few-shot CoT :** Providing examples that show the reasoning chain.

2.4 Self-Consistency

Definition : Self-Consistency is a technique used to improve the accuracy of Chain-of-Thought prompting by generating multiple reasoning paths and selecting the most frequent answer.

How it works :

- The same prompt is presented to the model multiple times (with temperature > 0 for diversity).

- Each generation produces a different reasoning chain and potentially a different answer.
- The final answer is determined by majority voting among all generated responses.
- This leverages the "wisdom of the crowd" principle for a single model.

Example :

- Generate 5 different reasoning chains for the same problem.
- If 3 out of 5 chains lead to "Entailment" and 2 lead to "Contradiction", select "Entailment".

Advantages :

- Significantly improves accuracy over single CoT generation.
- Reduces the impact of random errors or hallucinations.
- Particularly effective for complex problems where reasoning can vary.

Limitations :

- Requires multiple forward passes, increasing computational cost.
- May not work well if the model consistently makes the same type of error.
- Requires careful selection of the voting mechanism (simple majority vs. weighted).

2.5 Persona Prompting

Definition : Persona prompting involves instructing the model to adopt a specific role, expertise, or perspective when answering questions. This technique leverages the model's training on role-specific data.

How it works :

- The prompt explicitly assigns a role to the model (e.g., "You are a clinical expert", "Act as a doctor").
- The model adjusts its responses to match the knowledge and style associated with that role.
- This can improve domain-specific accuracy and response quality.

Example :

Prompt: "You are an experienced clinical researcher specializing in patient eligibility assessment. Classify the following patient-criterion relationship: [statement]"

Advantages :

- Improves domain-specific performance by activating relevant knowledge.
- Produces more professional and contextually appropriate responses.
- Can simulate expert-level reasoning and terminology.

Limitations :

- Effectiveness depends on the model's training data for that persona.
- May introduce biases associated with the chosen persona.
- Requires careful persona selection to match the task.

2.6 Inverse Interaction / Dialogue Simulation

Definition : Inverse interaction is a technique used to simulate patient-doctor dialogues or multi-turn conversations. Instead of providing direct answers, the model engages in a structured dialogue format.

How it works :

- The prompt sets up a conversational context (e.g., patient consultation).
- The model generates responses as if it were participating in a dialogue.
- Multiple turns can be simulated to explore different aspects of the problem.
- This is particularly useful for medical scenarios where information gathering is iterative.

Example :

Prompt: "Simulate a patient-doctor consultation:

Doctor: What brings you here today?

Patient: I'm experiencing chest pain.

Doctor: How long have you had this pain?

Patient: [model generates response]

Doctor: [model generates follow-up questions]"

Advantages :

- More natural and interactive problem-solving approach.
- Allows for iterative information gathering.
- Useful for complex scenarios requiring multiple pieces of information.
- Can reveal implicit information through dialogue.

Limitations :

- Requires careful prompt design to maintain conversation coherence.
- May generate irrelevant or off-topic dialogue turns.
- Higher computational cost due to multiple generation steps.
- May require post-processing to extract the final answer.